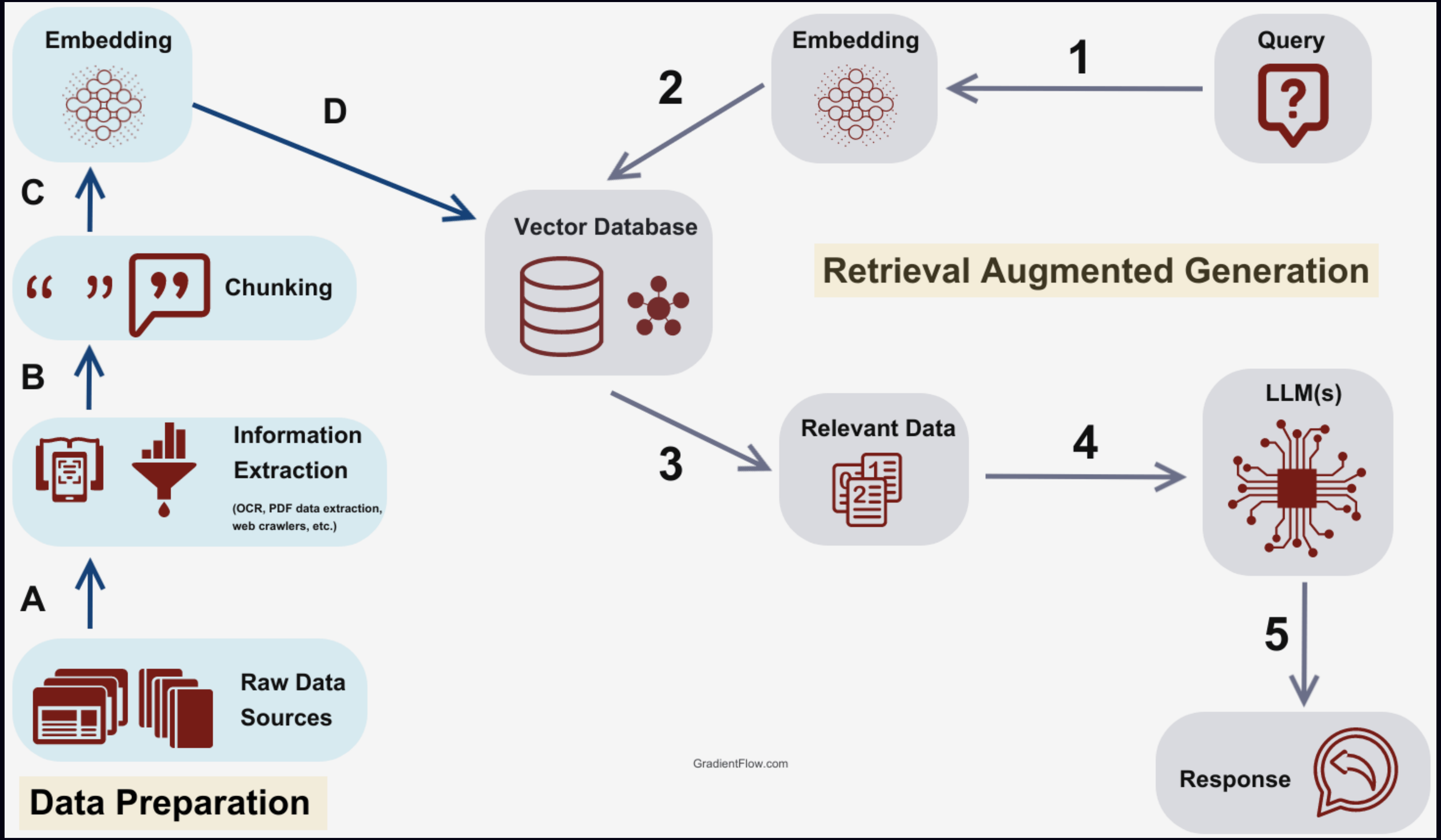
When Semantic Search Breaks

RAM Walls · Silent Failures · Architecture Decisions

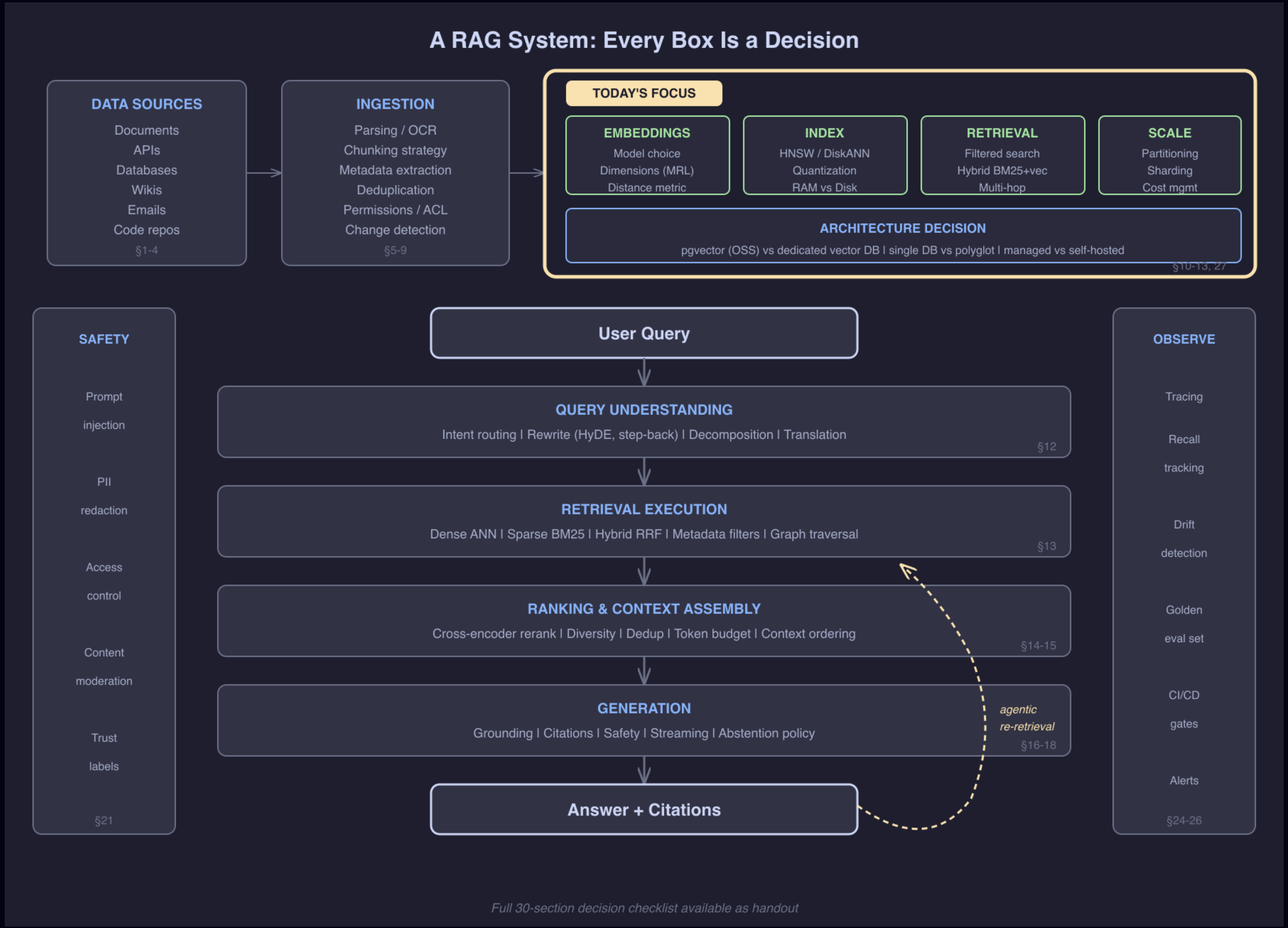
Jeevan

Open Source Summit Korea · August 2026

Entain



Embedding
D
C
Chunking
B
Information
Extraction
(OCR, PDF data extraction,
web crawlers, etc.)
A
Raw Data
Sources
Data Preparation
Embedding
2
1
Query
Vector Database
Retrieval Augmented Generation
3
Relevant Data
4
LLM(s)
5
Response
GradientFlow.com


A RAG System: Every Box Is a Decision
DATA SOURCES
Documents
APIs
Databases
Wikis
Emails
Code repos
§1-4
INGESTION
Parsing / OCR
Chunking strategy
Metadata extraction
Deduplication
Permissions / ACL
Change detection
§5-9
TODAY'S FOCUS
EMBEDDINGS
Model choice
Dimensions (MRL)
Distance metric
INDEX
HNSW / DiskANN
Quantization
RAM vs Disk
RETRIEVAL
Filtered search
Hybrid BM25+vec
Multi-hop
SCALE
Partitioning
Sharding
Cost mgmt
ARCHITECTURE DECISION
pgvector (OSS) vs dedicated vector DB | single DB vs polyglot | managed vs self-hosted
§10-13, 27
SAFETY
Prompt injection
PII redaction
Access control
Content moderation
Trust labels
§21
User Query
QUERY UNDERSTANDING
Intent routing | Rewrite (HyDE, step-back) | Decomposition | Translation
§12
RETRIEVAL EXECUTION
Dense ANN | Sparse BM25 | Hybrid RRF | Metadata filters | Graph traversal
§13
RANKING & CONTEXT ASSEMBLY
Cross-encoder rerank | Diversity | Dedup | Token budget | Context ordering
§14-15
GENERATION
Grounding | Citations | Safety | Streaming | Abstention policy
agentic re-retrieval
§16-18
Answer + Citations
OBSERVE
Tracing
Recall tracking
Drift detection
Golden eval set
CI/CD gates
Alerts
§24-26
Full 30-section decision checklist available as handout

The Scaling Path

We start from first principles and build up.

100K vectors
Everything works
Don't optimize.
Just ship.
SHIP IT

→

1-10M
Proper indexes
Need filters
Need MRL (256d)
GETTING REAL

→

10-50M
RAM wall hits
Need compression
Need quantization
CFO NOTICED

→

50M+
Disk indexes
Need DiskANN
Need observability
SILENT FAILURES

```
Month 1:  "Let's add semantic search!"
          → 100K vectors, works great ✅

Month 4:  "Scale to all our docs"
          → 10M vectors, still fine ✅

Month 8:  "Enterprise rollout"
          → 100M vectors, RAM bill explodes 💸

Month 9:  "Add per-tenant filtering"
          → recall silently drops to 40% 🔇

Month 10: "Maybe we need a separate vector DB?"
          → now syncing two databases forever 🔄
```

Same pattern. Same order. Every team.

This talk: the math, the tools, and the decisions — in plain English.

What We'll Cover

First Principles

Embeddings, distance, why approximate works

The RAM Wall

Where scale breaks and what to do

Three Compression Levers

Dimensions → Bits → Disk

Silent Failures

Filtered search + recall drift

Architecture Decisions

Benchmarks, trade-offs, decision matrix

🟦 Demo

▒▒ ✅ High Similarity

Same meaning, no shared words

"cancel my subscription"

"I want to unsubscribe"

→ ~68%

▒▒ ❌ Low Similarity

Same word, different meaning

"the bank is closed for the holiday"

"the river bank was steep and muddy"

→ ~26%

384 dimensions per sentence. That's what costs 920 GB at scale.

🟦 How Do Computers Compare Text?

Computers don't understand words.

```
"I love fries"   vs   "Fries are great"
```

👦 Human → instantly similar. 💻 Computer → two different strings.

The fix: turn text into numbers that capture meaning.

```
"I love fries"     → [0.2, 0.8, 0.1, ...] 384 numbers
"Fries are great"  → [0.3, 0.7, 0.2, ...] 384 numbers
"The sky is blue"  → [0.9, 0.1, 0.8, ...] 384 numbers
```

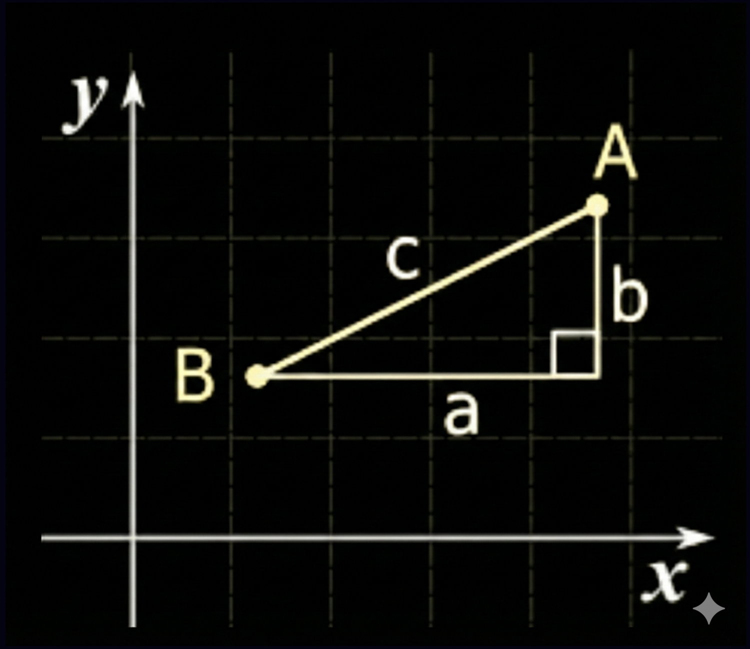
Similar meanings → similar numbers.

That's an embedding. A GPS coordinate for meaning — close meanings, close numbers.

How Do We Measure "Close"?

Distance between two points — already familiar:

```
Point A = (x₁, y₁)    Point B = (x₂, y₂)

Distance = √((x₂-x₁)² + (y₂-y₁)²)
```



Same idea, just more dimensions:

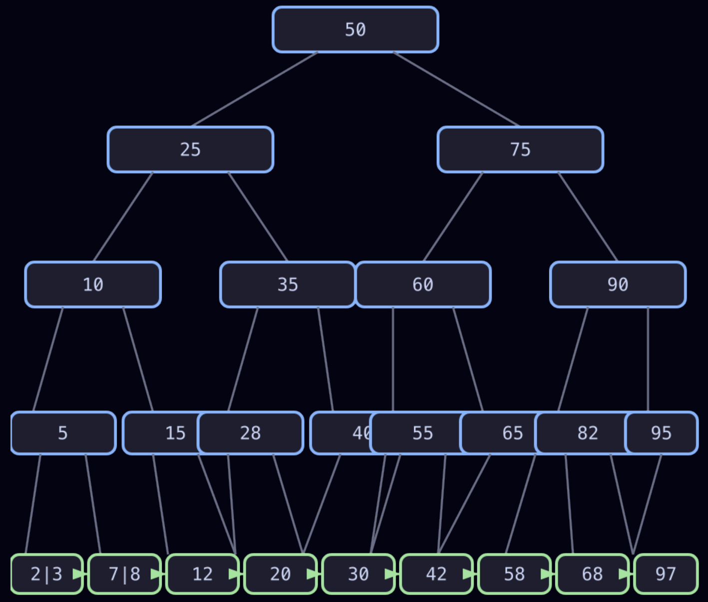
```
Vector A = [0.2, 0.8, 0.1, ... 384 nums]
Vector B = [0.3, 0.7, 0.2, ... 384 nums]

Distance = √((0.3-0.2)² + (0.7-0.8)² + ...)
```

That's Euclidean distance. Works, but...

How do you search when there's no sort order?

Why Vector Indexing Is Different

B-tree: exact ordering, binary search, O(log n)



B+ Tree — sorted keys, O(log n) lookup
Leaf-linked for range scans. Binary search works
because there's a natural order.

Vector indexes have no natural sort order.

```
A = [0.21, 0.87, 0.14, ..., 0.53]
B = [0.93, 0.12, 0.38, ..., 0.71]
C = [0.34, 0.76, 0.22, ..., 0.48]
```

No "less than" for 384 dimensions.

Exact search = check every vector — too slow at scale.

🟦 The Key Insight: Approximate Is Good Enough

What if we don't need the exact top 10?

What if finding 9 out of 10 true best matches is acceptable?

This is Approximate Nearest Neighbor (ANN) search.

```
Exact search:  ████████████████████ 100% scanned → 100% accurate → 🐌 Slow
ANN search:    ███░░░░░░░░░░░░░░░░░  ~5% scanned → ~95-99% accurate → ⚡ Fast!
```

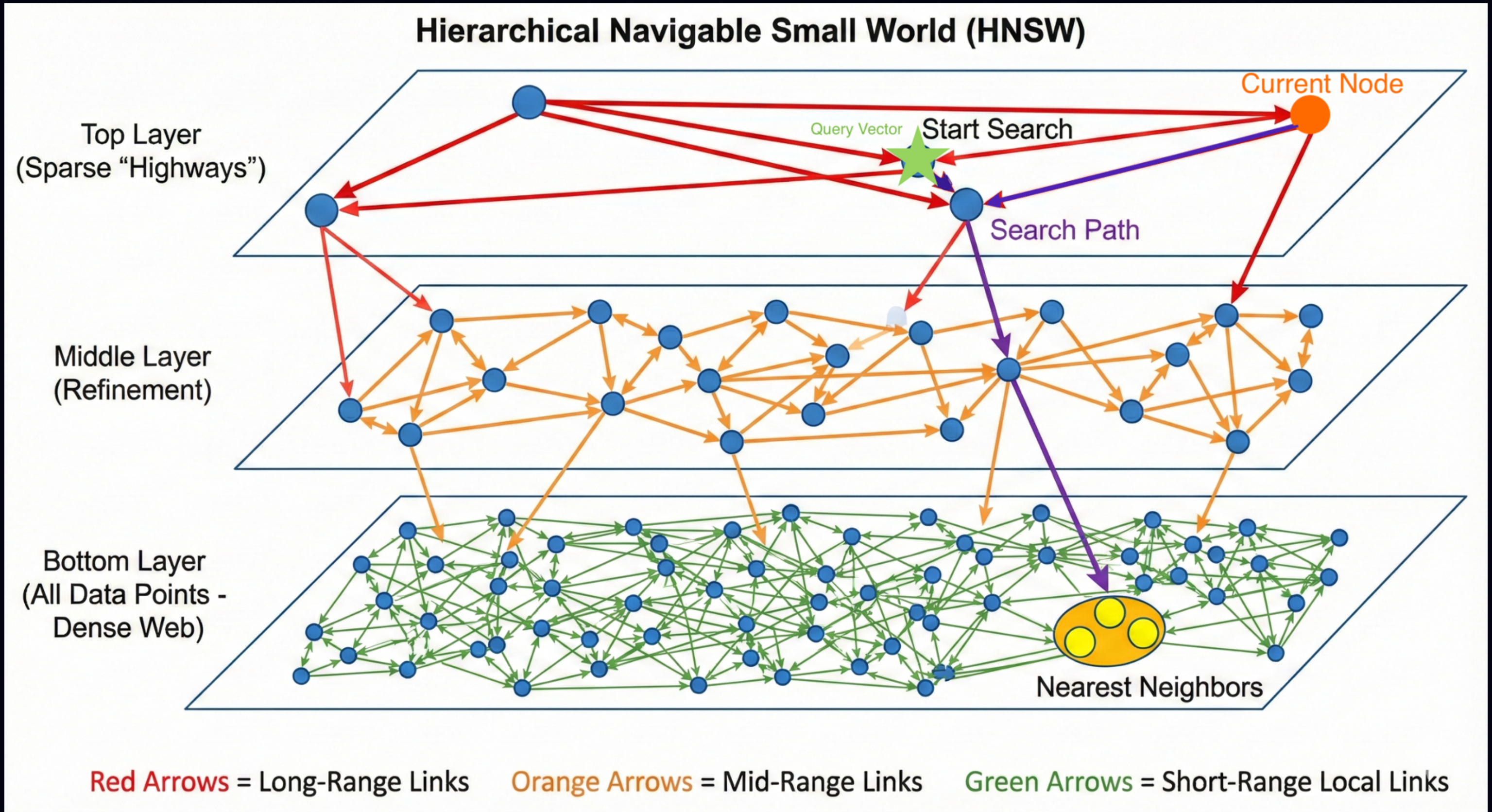
Analogy: 📦 Finding a delivery address in Seoul

- Exact search: Check every building in every street → hours 🐌
- With postal code: Go to the right 구(gu), check nearby blocks → minutes ⚡
- The catch: Might miss a building right on the border of two postal codes

This trade-off — 95% accuracy at 100x speed — is what makes vector search viable.

🟦 How ANN (Approximate Nearest Neighbor) Indexes Work

Close enough — without checking every vector.



HNSW — multi-layer graph · ~95-99% recall · 1-5ms · must stay in RAM

Like driving Seoul → Sokcho: expressway (top layer, big hops) → regional road → local street to the door (precise).

ANN = Approximate Nearest Neighbor · HNSW = Hierarchical Navigable Small World

What happens when your index outgrows RAM?

🟦 The RAM Wall

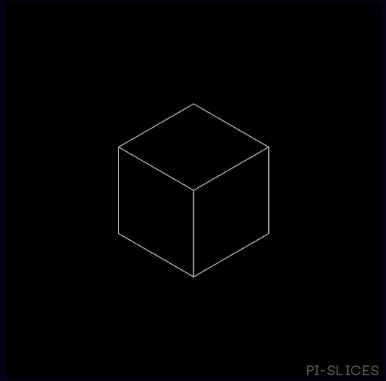
Everyone wants to build AI on their own data. Leadership wants to know why the infrastructure bill just tripled.

```
Per vector:  1536 dims × 4 bytes = 6,144 bytes ≈ 6 KB
```

Demo used 384d (MiniLM). Production models (OpenAI, Cohere) use 1536d. This is production math.

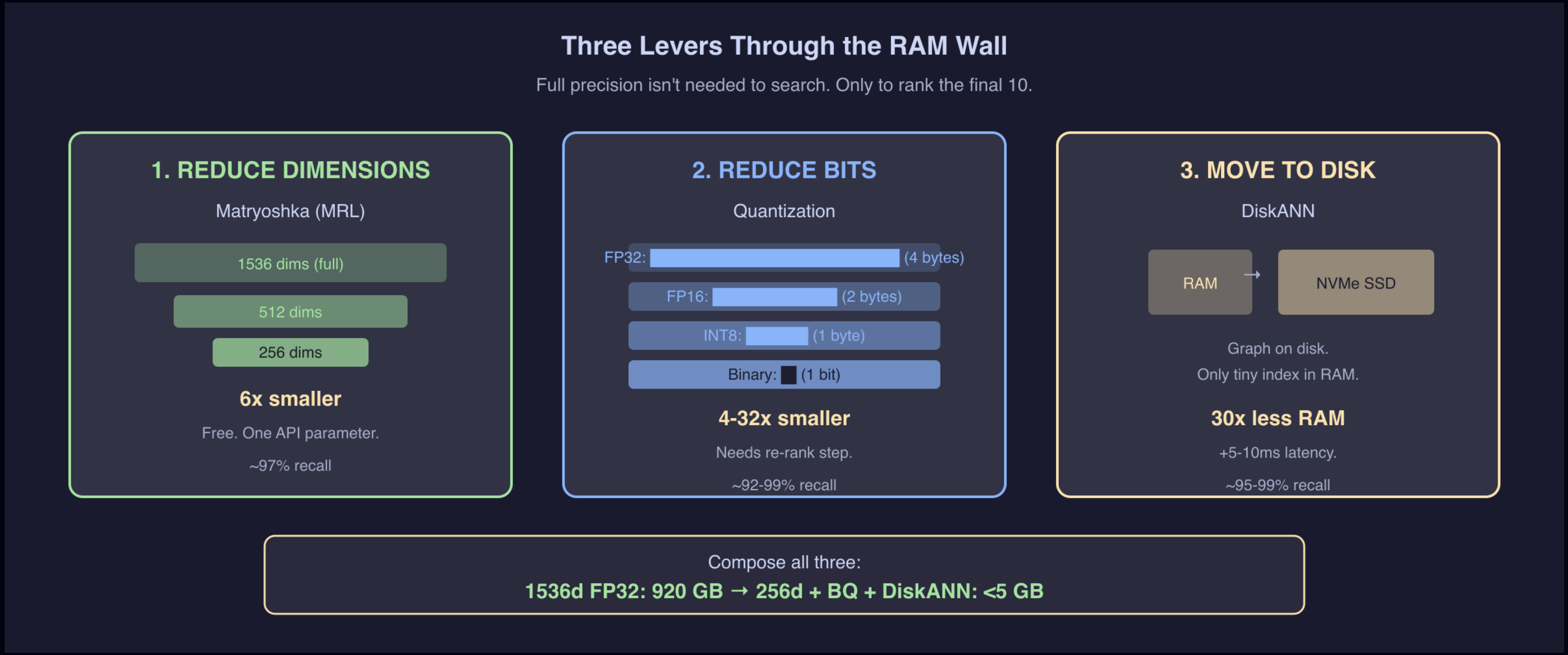
Scale	Raw Vectors	+ HNSW (50%)	Approx. RAM Cost
1M	6 GB	~9 GB	~$50/mo
10M	61 GB	~92 GB	~$500/mo
100M	614 GB	~920 GB	~$5,000+/mo
1B	6.1 TB	~9.2 TB	💀

64 GB → 920 GB = not 15x cost, it's 30-50x operational complexity.



1. "Just use HNSW" → 750 GB RAM → $5K/mo 💸
2. "Add a separate vector DB" → two systems to sync forever 🔄
3. "We'll optimize later" → index rebuild = 3 days, no deploys 🧊

Not bad tools — premature decisions made without doing the math.



920 GB → 5 GB compressed index. Full vectors stay on SSD for re-rank.

(Do them in order: dimensions first, then bits, then disk.)

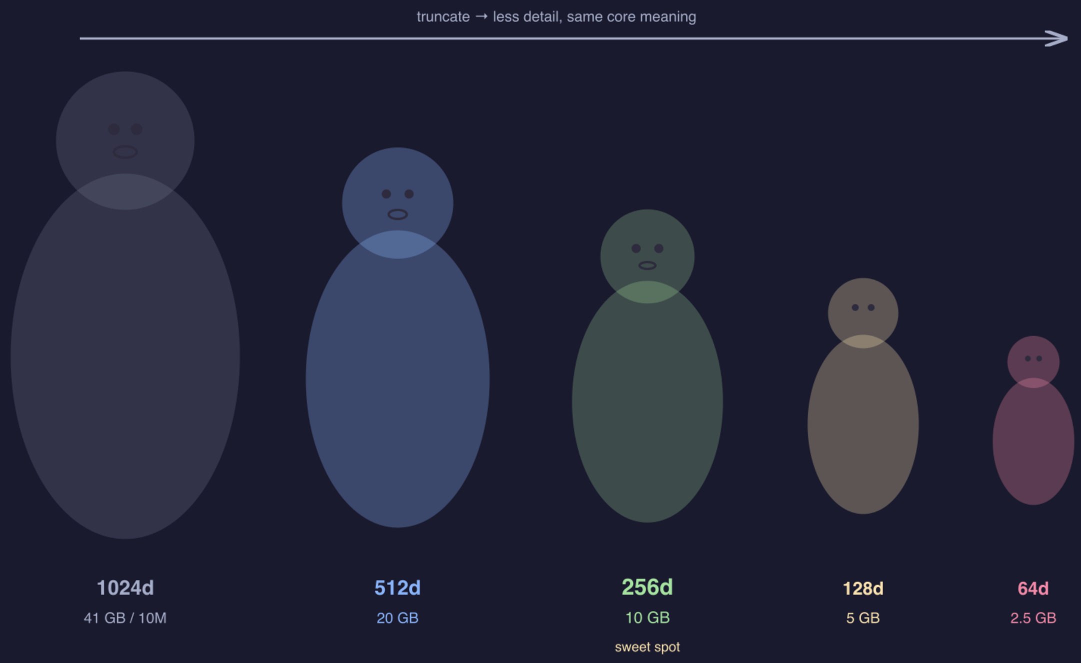
MRL = Matryoshka Representation Learning · SQ/PQ/BQ = Scalar/Product/Binary Quantization · DiskANN = disk-based ANN index

Same doll, fewer layers — still recognizable.

Same embedding, fewer dimensions — still useful.

Matryoshka Embeddings: Reduce Dimensions First

One model. Front dimensions carry the most meaning. Truncate safely.



How to use:

`embed("query", dimensions=256)`

One parameter change.

6x smaller. Free.

~97% recall at 256d

OpenAI, Cohere, Voyage, Jina, Nomic

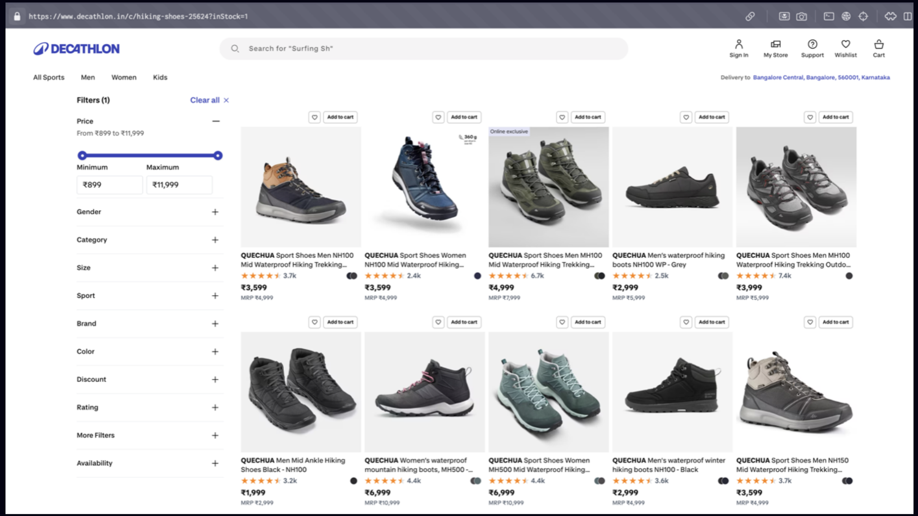
What each level captures:

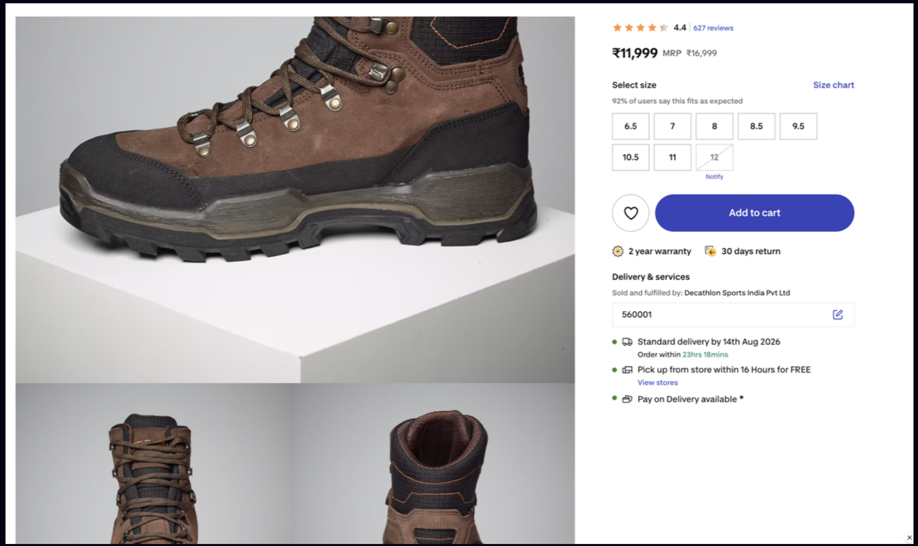
64d: "this is about refunds"

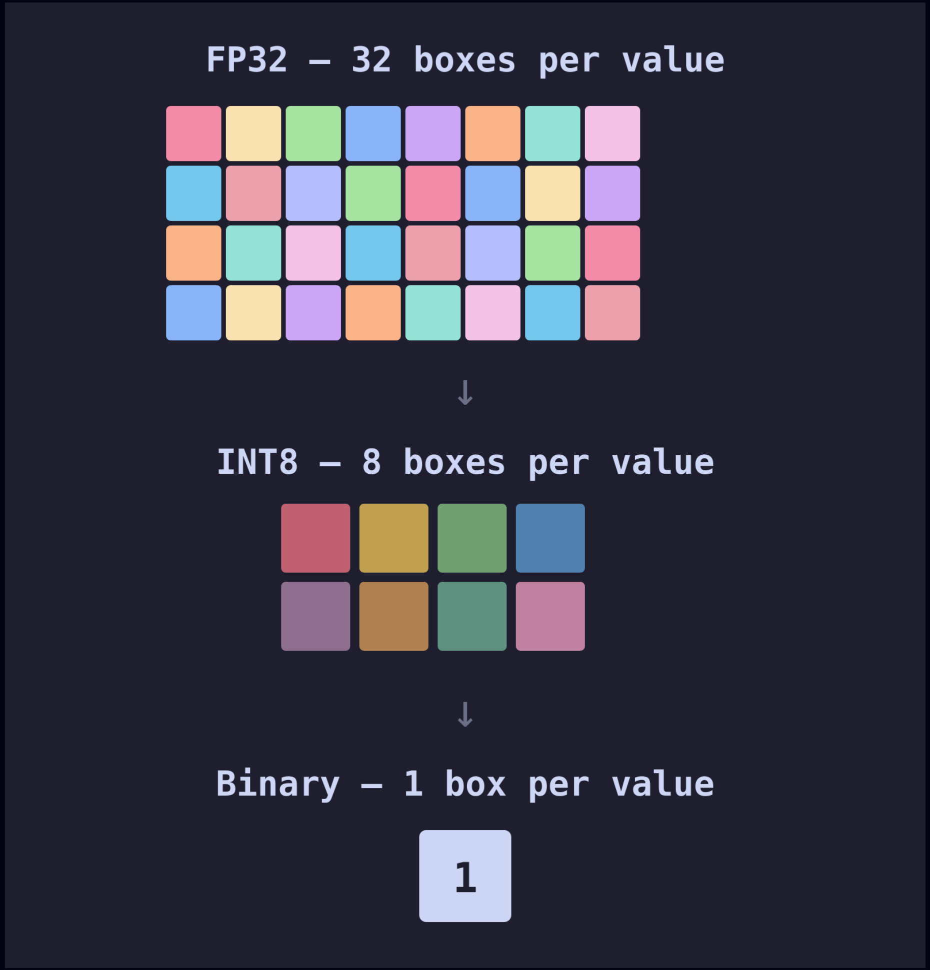
256d: "enterprise, 30-day policy"

1024d: "customer X, date Y, edge case"

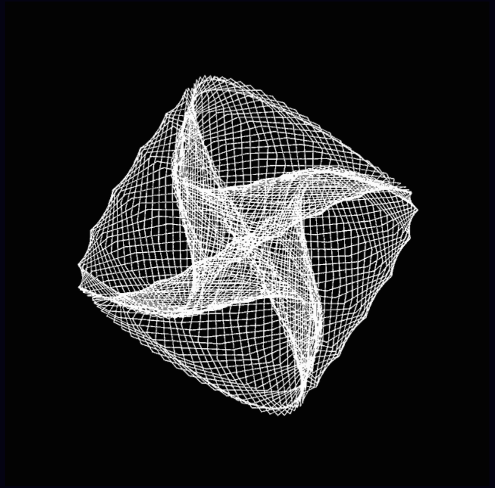
This is the first lever. Free. No infrastructure change. Do it first.


DECATHLON
Search for "Surfing Sh"
All Sports
Men
Women
Kids
Filters (1)
Clear all
Price
Minimum
Maximum
Gender
Category
Size
Sport
Brand
Color
Discount
Rating
More Filters
Availability
Add to cart
QUECHUA Sport Shoes Men NH100 Mid Waterproof Hiking Trekking...
₹3,599
QUECHUA Sport Shoes Women NH100 Mid Waterproof Hiking...
₹3,599
QUECHUA Sport Shoes Men MH100 Mid Waterproof Hiking Trekking...
₹4,999
QUECHUA Men's waterproof hiking boots NH100 WP - Grey
₹2,999
QUECHUA Sport Shoes Men MH100 Waterproof Hiking Trekking Outdo...
₹3,999
QUECHUA Men Mid Ankle Hiking Shoes Black - NH100
₹1,999
QUECHUA Women's waterproof mountain hiking boots, MH500 -...
₹6,999
QUECHUA Sport Shoes Women MH500 Mid Waterproof Hiking...
₹6,999
QUECHUA Men's waterproof winter hiking boots NH100 - Black
₹2,999
QUECHUA Sport Shoes Men NH150 Mid Waterproof Hiking Trekking...
₹3,599


4.4 627 reviews
₹11,999 MRP ₹16,999
Select size
Size chart
92% of users say this fits as expected
6.5
7
8
8.5
9.5
10.5
11
12
Notify
Add to cart
2 year warranty
30 days return
Delivery & services
Sold and fulfilled by: Decathlon Sports India Pvt Ltd
560001
Standard delivery by 14th Aug 2026
Order within 23hrs 18mins
Pick up from store within 16 Hours for FREE
View stores
Pay on Delivery available *


FP32 — 32 boxes per value
↓
INT8 — 8 boxes per value
↓
Binary — 1 box per value
1

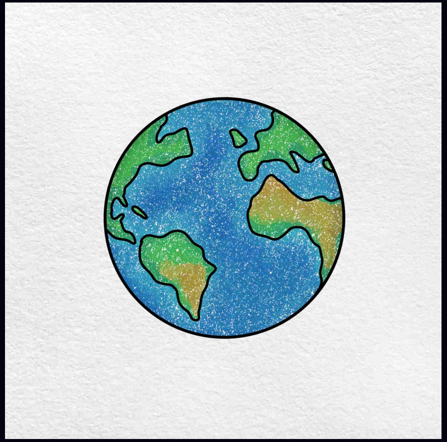

FP32 (32-bit) — 32 boxes per value
4 bytes/value
6 GB / 1M
$5K/mo at 100M
INT8 (8-bit) — 8 boxes per value
1 byte/value
1.5 GB / 1M
4x smaller · ~98%
Binary (1-bit) — 1 box per value
1
1 bit/value
192 MB / 1M
32x smaller
Memory at 100M vectors:
FP32: 614 GB
INT8: 153 GB
Binary: 19 GB
Search the blurry version fast.
Re-rank with the sharp version for accuracy.



Method	Size (1M)	Recall@10
FP32 (baseline)	6.1 GB	100%
Binary (1-bit)	192 MB	~10%
Binary + rerank	192 MB	~92-96%

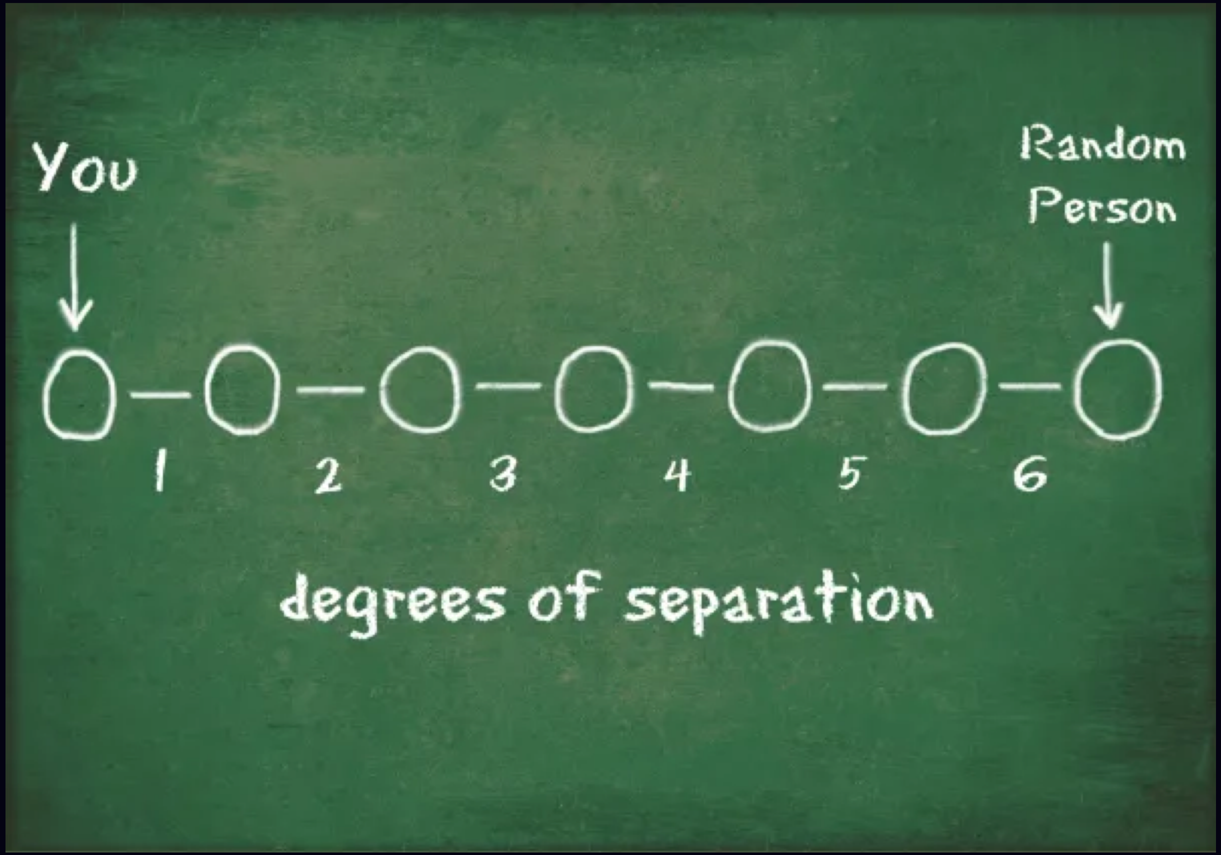
32× smaller. Still finds the right answers.

Recall is optimistic at small scale. At 1M+ vectors, expect ~92-96% with rerank — still excellent for most use cases.

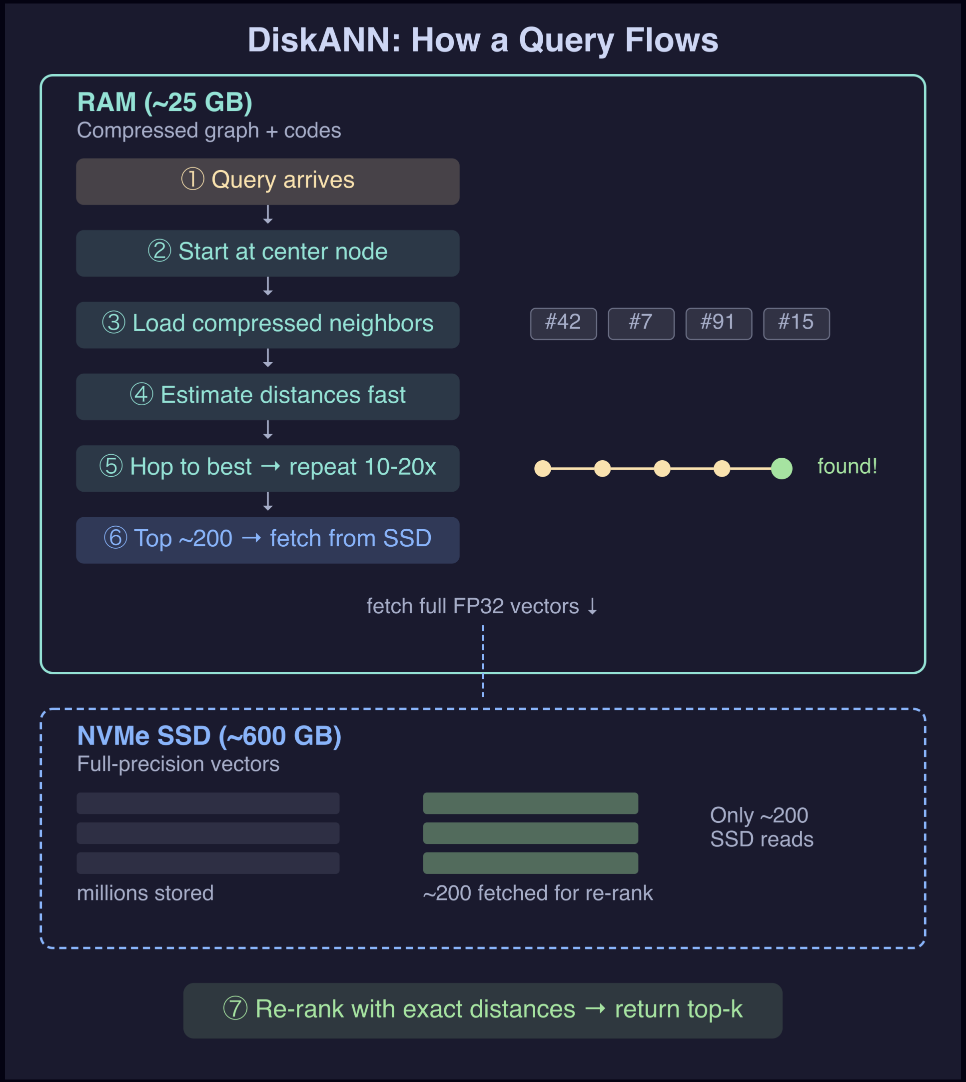


8 billion people on Earth.

Any two connected by just 6 hops.



DiskANN uses the same idea: build a graph where any vector is ~6 hops from any other. Keep the graph in RAM, keep the actual vectors on SSD.


DiskANN: How a Query Flows
RAM (~25 GB)
Compressed graph + codes
① Query arrives
② Start at center node
③ Load compressed neighbors
#42
#7
#91
#15
④ Estimate distances fast
⑤ Hop to best → repeat 10-20x
found!
⑥ Top ~200 → fetch from SSD
fetch full FP32 vectors ↓
NVMe SSD (~600 GB)
Full-precision vectors
millions stored
~200 fetched for re-rank
Only ~200
SSD reads
⑦ Re-rank with exact distances → return top-k

What can go wrong after you solve the wall?

🟦 Silent Failure #1: Filtered Search

```
SELECT * FROM products
WHERE tenant_id = 42
ORDER BY embedding <=> query LIMIT 10;
```

⚠️ HNSW returns 10 nearest vectors. Filter throws 9 away.

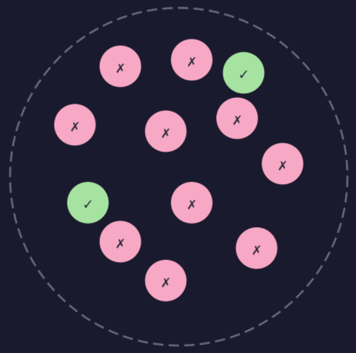
Asked for 10. Got 0.

No error. No warning. No log line.

Pre-Filter vs Post-Filter: Both Fail

Post-Filter (ANN → then filter)

HNSW graph (50K docs)



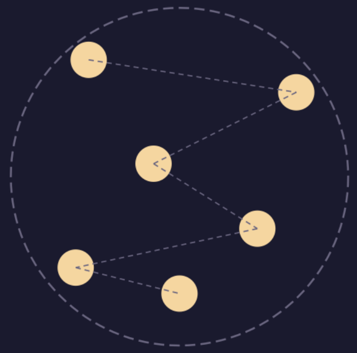
matches filter · doesn't match

ANN returns top 10 nearest
→ filter: only 2 match tenant_id=42

✗ Asked for 10, got 2

Pre-Filter (filter → then ANN)

HNSW graph (50K docs)



200 docs match tenant_id=42
→ scattered across graph

✗ Falls back to sequential scan

Both approaches break. You need adaptive filtered search.

Silent Failure #2: Recall Drift

#1 returns zero results. #2 returns the WRONG results.

```
Recall@10
 100% |  ●─────●
      |         \
  90% |          \
      |           \
  80% |            \
      |             \
  70% |              \
      |               \
  60% |                ●  ← nobody noticed
      |
      └──────────────────────
        Month 1   4     6     9
```

No error. No alert. No log line.

The system doesn't crash. It just quietly becomes useless.


Detecting Drift: Two Monitoring Flows
Canary: Mean Distance to Results
Metric: mean cosine distance | Farther = less relevant
0.40
0.30
0.20
0.10
threshold
Week 1
Week 4
Week 8
Week 12
0.15
0.21
0.29
0.35
Results drifting away from queries.
Log on every search. No ground truth needed.
Audit: Recall@10 Over Time
Metric: recall@10 | 200 questions with known answers
100%
95%
90%
85%
80%
95% target
Week 1
Week 4
Week 8
Week 12
96%
92%
84%
80%
Quality dropping silently.
Run weekly. Definitive ground truth.
If you don't measure weekly, you're flying blind.

SCALE →

< 1M	1M – 10M	10M – 50M	50M+
pgvector + HNSW	+ MRL (256d)	+ Quantization	DiskANN
Just ship.	6× free savings	32× smaller	RAM → SSD

Start with what you have. Stay longer than you think.

Exhaust: dimensions → quantization → DiskANN
before adding infrastructure.

🟦 The End

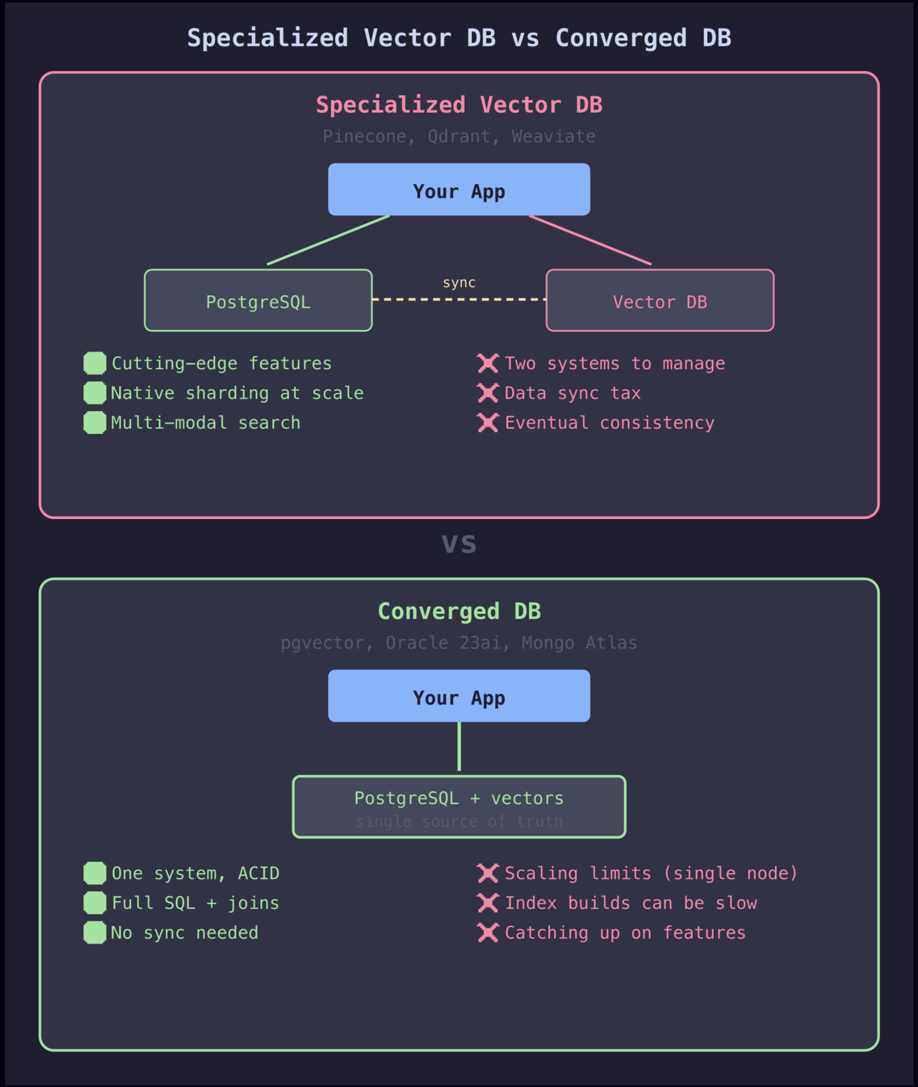
Remember Month 10? You don't have to get there.

Do the math early. Pick compression before it's urgent. Monitor recall always.

Scan for the repo & slides

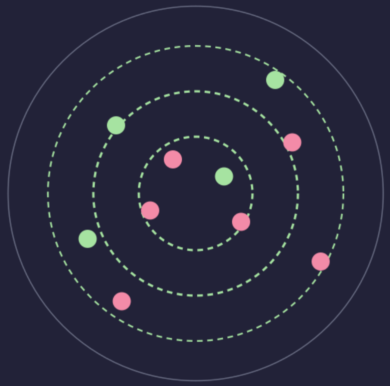
📫 hello@noobj.me

🌐 noobj.me


Specialized Vector DB vs Converged DB
Specialized Vector DB
Pinecone, Qdrant, Weaviate
Your App
PostgreSQL
sync
Vector DB
Cutting-edge features
Native sharding at scale
Multi-modal search
Two systems to manage
Data sync tax
Eventual consistency
VS
Converged DB
pgvector, Oracle 23ai, Mongo Atlas
Your App
PostgreSQL + vectors
single source of truth
One system, ACID
Full SQL + joins
No sync needed
Scaling limits (single node)
Index builds can be slow
Catching up on features

1. Iterative Scan

Keep scanning until enough match



matching tenant   other tenants

Scan 40 → not enough → scan 40 more

Works for any filter ✓

pgvector 0.8+

Best for: high cardinality (10K+ values)

2. Partial Index

Separate graph per filter value

One HNSW graph per category

Zero wasted traversal ✓

CREATE INDEX ... WHERE category = 'X'

Best for: low cardinality (2-100 values)

3. Table Partitioning

One partition + index per tenant

Tenant A (+ own HNSW)

Tenant B (+ own HNSW)

Tenant C (+ own HNSW)

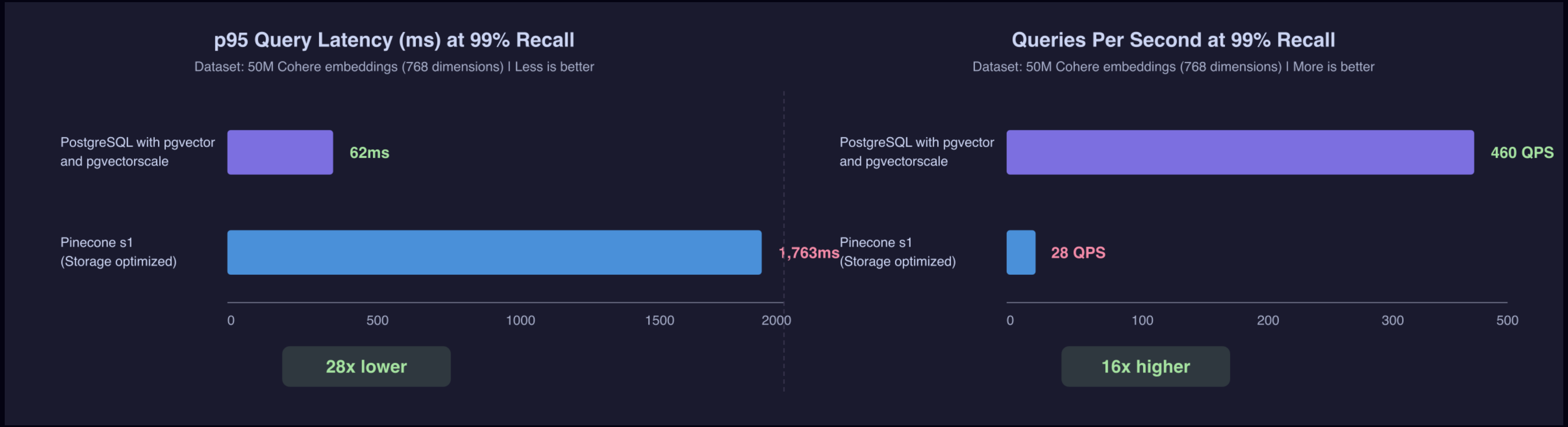
Tenant D ...

Storage-level isolation

Scales to thousands ✓

PARTITION BY LIST (tenant_id)

Best for: multi-tenant SaaS



50M Cohere embeddings | 768 dimensions | 99% recall | Same AWS hardware

pgvector + pgvectorscale: 28x faster, 16x more throughput, 75% less cost

Usually one Postgres wins. Dedicated engines earn their keep only at extreme scale — measure first.

Source: github.com/timescale/pgvectorscale | All open source (PostgreSQL License + Apache 2.0)

Appendix: Hybrid Search — BM25 + Vectors

Vector search misses exact terms. Keyword misses meaning.

```
┌──────────────────────────────────────────────────────────────┐
│ Query:  "how to handle payment refund timeout"               │
├──────────────────────────────────────────────────────────────┤
│ BM25      →  matches "refund timeout"        (precise)       │
│ Vector    →  finds "payment reversal logic"  (broader)       │
│ Combined  →  best recall                                     │
└──────────────────────────────────────────────────────────────┘
```

Both are PostgreSQL extensions — one DB, one query.

No separate service · no sync · same transaction.

BM25 = keyword ranking · RRF = Reciprocal Rank Fusion

Appendix: Quantization Trade-offs

	FP32	FP16 (half)	Scalar INT8	Product (PQ)	Binary (BQ)
Compression	1x	2x	4x	8-64x	32x
Recall (no re-rank)	100%	~99.9%	~95-98%	~85-95%	~75-95%*
Recall (w/ re-rank)	—	—	~98-99%	~95-99%	~92-96%
Speed vs FP32	1x	~1.5x	~2-3x	~5-10x	~15-30x
Training needed?	No	No	No	Yes	No
Best for	Small scale	Easy first win	General purpose	Massive datasets	Speed-critical

MRL + quantization compose: 256d + BQ = 192x compression from 1536d FP32.

FP16 (halfvec) · 2x compression

```
ALTER TABLE docs ADD COLUMN embedding_half halfvec(1536);
UPDATE docs SET embedding_half = embedding::halfvec(1536);
CREATE INDEX ON docs USING hnsw (embedding_half halfvec_cosine_ops);
```

Binary Quantization · 32x compression + re-rank

```
ALTER TABLE docs ADD COLUMN embedding_bit bit(1536);
UPDATE docs SET embedding_bit = binary_quantize(embedding)::bit(1536);
CREATE INDEX ON docs USING hnsw (embedding_bit bit_hamming_ops);

WITH candidates AS (
  SELECT id, embedding FROM docs
  ORDER BY embedding_bit <~> binary_quantize($query)::bit(1536) LIMIT 200
)
SELECT id, content FROM candidates ORDER BY embedding <=> $query LIMIT 10;
```

DiskANN + SBQ (pgvectorscale) · automatic

```
CREATE INDEX ON docs USING diskann (embedding vector_cosine_ops)
  WITH (num_neighbors = 50, storage_layout = 'memory_optimized');
```

```sql
CREATE EXTENSION IF NOT EXISTS vectorscale;

CREATE INDEX ON docs
  USING diskann (embedding vector_cosine_ops)
  WITH (num_neighbors = 50, search_list_size = 100);

-- Query unchanged — planner picks diskann
SELECT id, content FROM docs
ORDER BY embedding <=> $query LIMIT 10;

-- Tune at query time
SET diskann.query_search_list_size = 150;
```

Signal	Action
RAM usage > 60% of instance	Evaluate DiskANN
Dataset > 20-50M vectors	DiskANN likely cheaper
Latency budget allows 5-15ms	DiskANN is a fit

```sql
WITH bm25 AS (
  SELECT id, ROW_NUMBER() OVER (ORDER BY paradedb.score(id) DESC) AS rank
  FROM docs
  WHERE content @@@ 'payment refund timeout'
  LIMIT 100
),
vector AS (
  SELECT id, ROW_NUMBER() OVER (ORDER BY embedding <=> $query) AS rank
  FROM docs
  ORDER BY embedding <=> $query
  LIMIT 100
)
SELECT COALESCE(b.id, v.id) AS id,
       COALESCE(1.0/(60 + b.rank), 0)
     + COALESCE(1.0/(60 + v.rank), 0) AS rrf_score
FROM bm25 b
FULL OUTER JOIN vector v ON b.id = v.id
ORDER BY rrf_score DESC
LIMIT 10;
```

k=60 from the RRF paper — dampens high-rank dominance. Only cares about rank position, not raw scores.

Appendix: Recall Drift — Observability Setup

Two monitoring flows:

	Live Queries (canary)	Golden Eval (audit)
Frequency	Every query	Weekly
Measures	Distance distribution	Actual recall@10
Ground truth?	No	Yes
Catches	Gross failures fast	Subtle degradation

Minimum viable setup:

1. Log `mean_distance` on every search call
2. 200 labeled query→doc pairs, run weekly
3. Alert if recall drops below 95%
4. CI gate: fail deploy if recall regresses

Open source eval tools: RAGAS, Phoenix (Arize), Langfuse

Appendix: Long Context vs RAG

Factor	Favors Long Context	Favors RAG
Corpus size	<100K tokens	>100K tokens
Relevance ratio	>20% relevant	<20% relevant
Latency SLO	Async (45s ok)	Interactive (<2s)
Data freshness	Static	Frequently updated
Query volume	Hundreds/month	Thousands/day
Cost/query	$0.20–$2.00	$0.00008

1,250x cost difference at scale. If any TWO factors favor RAG → use RAG.

References

1. pgvector — github.com/pgvector/pgvector (PostgreSQL License)
2. pgvectorscale — github.com/timescale/pgvectorscale (Apache 2.0)
3. DiskANN — github.com/microsoft/DiskANN (MIT)
4. ParadeDB — github.com/paradedb/paradedb (AGPL)
5. Matryoshka Embeddings — arxiv.org/abs/2205.13147
6. pgvector 50M Benchmark — github.com/timescale/pgvectorscale/blob/main/BENCHMARKS.md
7. RAGAS — github.com/explodinggradients/ragas (Apache 2.0)
8. BGE-Reranker — huggingface.co/BAAI/bge-reranker-v2-m3
9. Embedding Quantization — huggingface.co/blog/embedding-quantization
10. Long Context vs RAG — tianpan.co/blog/2026-04-09
11. ANN Benchmarks — ann-benchmarks.com
12. LangGraph — github.com/langchain-ai/langgraph (MIT)